AGENTIC REVENUE SYSTEM

EXECUTIVE AND TECHNICAL OVERVIEW

Prepared: October 12, 2026

EXECUTIVE SUMMARY

The Agentic Revenue System is an AI-assisted revenue operations platform. It supports the lifecycle from lead intake and qualification through scoring, outreach, reply handling, call intelligence, IO generation, human approval, external delivery, signature workflow, and audit.

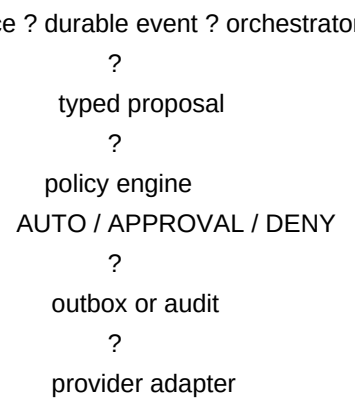
The governing principle is: AI proposes; deterministic software validates and governs; humans approve high-impact actions; adapters perform external side effects; audit records what happened.

END-TO-END EXECUTION

1. A lead enters through a CRM, import, webhook, or fixture. The current demo uses fixtures/leads.json; the system does not yet autonomously discover leads from the open internet.
2. The lead becomes a durable lead.created event with an idempotency key.
3. The orchestrator creates or updates a deal in SQLite.
4. The segment registry loads the brand or publisher pack.
5. The deterministic scoring skill calculates fit, intent, engagement, total score, tier, explanation, and fired rules.
6. An agent produces a typed proposal such as an outreach draft, reply classification, transcript digest, or IO draft.
7. The policy engine classifies the action as AUTO, REQUIRE_APPROVAL, or DENY.
8. High-impact actions enter the approval queue.
9. A human approves or rejects the action.
10. Approved actions enter the durable outbox.
11. The delivery service selects the provider adapter and performs the external action.
12. The action is marked sent only after provider success; the provider ID and result are audited.

ARCHITECTURE

text



The project is a governed modular monolith. It avoids unnecessary microservices while keeping domain, orchestration, governance, agents, skills, segments, and adapters separate.

LEAD INTAKE AND CLASSIFICATION

The lead comes into the system from an external source. In the demo, fixture records simulate this source. A future CRM or prospecting connector can emit the same normalized event.

Classification uses deterministic business inputs such as company fit, employee count, locations, buying signals, pricing-page activity, monthly impressions, inventory types, and engagement. The segment pack supplies segment-specific fields and guidance.

Scoring is implemented in src/revenue_system/skills/scoring.py. The orchestrator applies it, and src/revenue_system/domain/models.py stores the resulting Score.

Scoring answers: How qualified or valuable is this lead?

Policy answers: What is the system allowed to do with this lead?

The LLM is not the final scoring authority. Delegating scoring entirely to an LLM would make results less repeatable, harder to explain and test, and more vulnerable to prompt injection. The LLM may interpret unstructured context, but application code calculates the final score and tier.

SEGMENT PACKS

A segment pack is a configuration package for one business segment. Current packs are packs/brand/ and packs/publisher/.

Packs can define qualification fields, messaging guidance, scoring inputs, and templates. The same agents and orchestrator serve both segments. Adding a segment should normally require a new pack rather than copied application logic.

Pack loading and validation are implemented in src/revenue_system/segments/registry.py and schema.py.

DURABLE STATE

The workflow does not depend on in-memory state. SQLite stores deals, events, approvals, outbox records, audit records, lessons, and system flags. This protects the workflow from restarts, crashes, deployments, retries, and duplicate webhooks.

SQLite is appropriate for this demo. PostgreSQL is recommended for production scale, concurrent workers, and high availability.

WHY AGENTS DO NOT DIRECTLY SEND EMAIL

Agents are responsible for interpretation and proposal generation, not business side effects. They do not directly send email, change CRM data, change pricing, or send contracts.

This separation provides human approval, deterministic validation, prompt-injection protection, provider independence, and complete auditability.

text
Agent ? proposes
Policy ? decides
Human ? approves high-impact actions
Outbox ? stores approved work
Adapter ? performs provider side effect
Audit ? records the result

OUTBOX DELIVERY

src/revenue_system/orchestration/delivery.py performs outbox delivery. It is invoked with:

powershell
python -m revenue_system.cli approvals deliver

It reads pending rows, selects the adapter by action type, sends the request, uses an idempotency key, marks the item sent only after success, and audits failures without losing pending work.

Current paths are:

text
OutboxDeliveryService ? MailAdapter ? Resend
OutboxDeliveryService ? DropboxSignAdapter ? Dropbox Sign

AGENTS AND DETERMINISTIC SKILLS

Agents handle outreach, reply triage, call intelligence, deal desk/IO drafting, research boundaries, and pipeline analysis. They return typed Pydantic models.

Deterministic skills handle scoring, policy, state transitions, IO validation, templates, tracking, lessons, and idempotency-sensitive decisions.

TOOLS USED

? Python: application language.
? Pydantic/Pydantic Settings: typed contracts and configuration.
? SQLite: local durable operational store.
? Typer: operational CLI.
? Jinja2: email and document templates.
? OpenRouter: current live LLM provider using a structured Claude model.
? Claude Agent SDK: supported alternative LLM runner.
? Resend: controlled real email delivery.
? Dropbox Sign: e-signature adapter; test mode is used without a paid plan.
? FastAPI/Uvicorn: webhook and service foundation.
? Git/GitHub: version control and collaboration.

SAFETY AND GOVERNANCE

The system includes human approval for Tier-A outreach and commercial actions, prompt-injection detection, typed schemas, deterministic validation, idempotency, recipient allowlisting, explicit live side-effect control, kill switch, provider error handling, and audit records.

RUNTIME MODES

Mode	Purpose
test	fake runners and isolated tests
replay	deterministic offline demo
record	capture live responses for replay
live	configured provider adapters

Modes are selected independently per integration.

CURRENT VERIFICATION

The project has verified 37 passing tests, both segment packs, live OpenRouter execution, real allowlisted Resend delivery, approval and rejection, approved outbox delivery, audit provider IDs, prompt-injection handling, deterministic IO repair, and Dropbox Sign test-mode request creation.

DROPBOX SIGN WITHOUT A PAID PLAN

ESIGN_TEST_MODE=true creates a test request and records its provider ID, but does not send a real signer notification. The included fixtures/demo-agreement.pdf is fictional and intended only for testing.

PRODUCTION RECOMMENDATIONS

Before large-scale operation, migrate SQLite to PostgreSQL, supervise workers, use a secrets manager, configure signed webhooks, add monitoring and rate limits, configure backups and credential rotation, enforce real domain governance, and obtain an approved paid e-signature plan before disabling test mode.

OPERATIONAL COMMANDS

```
powershell
python -m revenue_system.cli validate-packs
python -m revenue_system.cli demo --mode replay
python -m revenue_system.cli demo --mode live
python -m revenue_system.cli approvals list
```

```
python -m revenue_system.cli approvals approve <approval-id>
python -m revenue_system.cli approvals reject <approval-id>
python -m revenue_system.cli approvals deliver
python -m revenue_system.cli audit <deal-id>
python -m revenue_system.cli kill-switch --enabled
python -m pytest -q
```